

Temperature Data Analysis API Assignment

Gowtham S, EE24B017

September 2025

Introduction

This report describes the implementation of several analytical functions for processing and interpreting historical temperature data from the Berkeley Earth dataset. The focus is on clarity and correctness of the approach, while ensuring that missing data and edge cases are handled gracefully.

`find_temperature_extremes`

I retrieve the temperatures of the target city using the `get_city_temperatures` function, and if it returns an empty dictionary, it means the city was not present. In that case, I raise a `ValueError`. Next, I initialize four variables to hold the maximum temperature, the date of the maximum, the minimum temperature, and the date of the minimum. I then iterate through the dictionary and compare values to update the maximum and minimum as required.

`get_seasonal_averages`

I check for the presence of the required city. Then I define the seasons and their corresponding months. By iterating through the dictionary, I extract the data for the required months. I check if I have atleast 1 datapoint. Otherwise, I calculate the seasonal averages and return the result.

`compare_decades`

I check for the presence of the required city. I then validate if the given decade inputs are valid. Next, I initialize the average and count variables for each decade. I iterate through the dictionary and add the temperatures that belong to the required decades. After verifying that datapoints exist for both decades, I compute the averages and return the result. If 1 decade has no datapoints, I return average as 'NA' for that decade, and difference and trend also as 'NA', but I compute the average for the other decade which does have some datapoints.

`find_similar_cities`

First, I wrote a helper function to load in the data of all cities at once. I calculate the average temperature of each city and store it in a dictionary with city names as the keys and their average temperatures and countries as values. I then check if the target city is present in this list of cities. Next, I iterate through the dictionary and identify cities with similar average temperatures. If the difference between their averages falls within the tolerance, I add them to another dictionary called `similar`. Finally, I return the result.

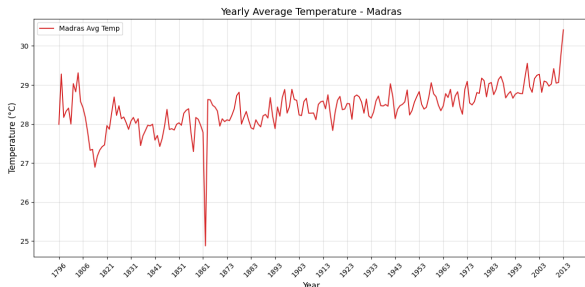
get_temperature_trends

I get the monthly average temperature data. I then calculate yearly averages and store them in `raw_annual_data`. After sorting the years into `sorted_years`, I collect the corresponding annual averages in `annual_values`. To smooth out fluctuations, I use a centered moving average (CMA). CMA is better suited for historical data than a simple moving average (SMA) because it doesn't shift the center like a SMA, which just calculates average of past `window_size` years. For odd window sizes, I take the centre year along with equal numbers of years before and after; for even window sizes, I shift the window slightly backward to maintain integer year values. At the dataset edges, the averaging window is reduced accordingly. I considered excluding edge years entirely, but that reduced the number of points significantly (e.g., from 151 to 136 for Abidjan). Therefore, I included edge years.

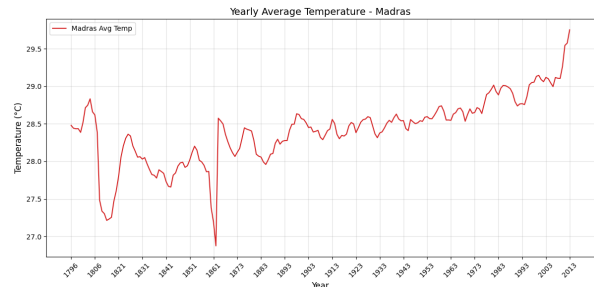
Next, I calculate the overall slope using linear regression, fitting a line through all points. This slope provides an estimate of the long-term warming or cooling trend. To identify warming and cooling periods, I rely on the moving average data rather than raw annual values, since the latter contains high variation. The moving averages highlight clearer periods of 5–6 years, compared to the noisy 1–2 year fluctuations in raw data. I then track the direction of the trend while iterating through the years, resetting whenever the trend changes. At the end, I handle the last year manually to ensure completeness. Finally, I return the result.

demo_plot

This function plots the temperature of a particular city using matplotlib. It has 2 modes, 1 using raw annual data and the other uses centered moving averages with a 5-year window. Instead of plotting all the years on the x-axis, I instead mark labels every 10 years or so (it varies sometimes due to missing data).



(a) Using raw annual data (Mode 0)



(b) Using Centered moving averages (Mode 1)

Figure 1: Plot of Average Temperature vs years

Conclusion

The implemented functions collectively provide a systematic approach to analyzing temperature data for cities worldwide. The methods are simple but effective, and they ensure meaningful insights while handling data limitations properly.